

DATA ENGINEERING PREPARATION

DAY 9 EXERCISES

LINUX PRACTICE

Linux Task 1 - awk basics

1. Print only the appt_id and fee columns.

```
awk -F',' '{print $1, $6}' appointments.csv
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ awk -F',' '{print $1 $6}' appointments.csv
appt_idfee
1500
2200
3750
4600
5300
6200
7900
8400
9150
10850
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ |
```

2. Print total fee collected across all rows.

Skip the header.

```
awk -F',' 'NR>1 {sum += $6} END {print sum}'
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ awk -F',' 'NR>1 {sum += $6} END {print sum}' appointments.csv
4850
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ |
```

3. Print appointments where fee is above 500.

```
awk -F',' 'NR>1 && $6 > 500 {print $0}'
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ awk -F',' 'NR>1 && $6 > 500 {print $0}' appointments.csv
3,3,3,2024-01-12,Knee pain evaluation,750
4,4,2,2024-01-13,Migraine treatment,600
7,6,1,2024-01-16,Cardiac stress test,900
10,2,2,2024-01-19,Epilepsy management,850
prashant@DESKTOP-B9KE2N7:/mnt/d/Kode-x notes/linux$ |
```

4. Print the count of rows where

doctor_id is 5.

```
awk -F',' '$3 == 5 {count++} END {print count}'
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ awk -F',' '$3 == 5 {count++}END {print count}' appointments.csv
3
```

5. Print average fee across all appointments.

```
awk -F',' 'NR>1 {sum+=$6; count++} END {print "Avg Fee:", sum/count}'
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ awk -F',' 'NR>1 {sum+=$6; count++} END {print "Average fee:" sum/count}' appointments.csv
Average fee:485
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

Linux Task 2 - sed basics

1. Replace "Hypertension checkup" with

"BP checkup" in appointments.csv.

Print the result only — do not modify file.

sed 's/Hypertension checkup/BP checkup/'

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sed 's/Hypertension checkup/BP checkup/g' appointments.csv
appt_id,patient_id,doctor_id,appt_date,diagnosis,fee
1,1,1,2024-01-10,BP checkup,500
2,2,5,2024-01-11,Fever and cold,200
3,3,3,2024-01-12,Knee pain evaluation,750
4,4,2,2024-01-13,Migraine treatment,600
5,5,4,2024-01-14,Acne treatment,300
6,1,5,2024-01-15,Follow-up general checkup,200
7,6,1,2024-01-16,Cardiac stress test,900
8,7,3,2024-01-17,Back pain physiotherapy,400
9,8,5,2024-01-18,Routine vaccination,150
10,2,2,2024-01-19,Epilepsy management,850
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$
```

2. Delete the header line from output.

sed '1d' appointments.csv

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sed '1d' appointments.csv
1,1,1,2024-01-10,Hypertension checkup,500
2,2,5,2024-01-11,Fever and cold,200
3,3,3,2024-01-12,Knee pain evaluation,750
4,4,2,2024-01-13,Migraine treatment,600
5,5,4,2024-01-14,Acne treatment,300
6,1,5,2024-01-15,Follow-up general checkup,200
7,6,1,2024-01-16,Cardiac stress test,900
8,7,3,2024-01-17,Back pain physiotherapy,400
9,8,5,2024-01-18,Routine vaccination,150
10,2,2,2024-01-19,Epilepsy management,850
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

3. Print only lines 3 to 7.

sed -n '3,7p' appointments.csv

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sed -n '3,7p' appointments.csv
2,2,5,2024-01-11,Fever and cold,200
3,3,3,2024-01-12,Knee pain evaluation,750
4,4,2,2024-01-13,Migraine treatment,600
5,5,4,2024-01-14,Acne treatment,300
6,1,5,2024-01-15,Follow-up general checkup,200
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

4. Remove all lines that contain "vaccination".

sed '/vaccination/d' appointments.csv

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sed '/vaccination/d' appointments.csv
appt_id,patient_id,doctor_id,appt_date,diagnosis,fee
1,1,1,2024-01-10,Hypertension checkup,500
2,2,5,2024-01-11,Fever and cold,200
3,3,3,2024-01-12,Knee pain evaluation,750
4,4,2,2024-01-13,Migraine treatment,600
5,5,4,2024-01-14,Acne treatment,300
6,1,5,2024-01-15,Follow-up general checkup,200
7,6,1,2024-01-16,Cardiac stress test,900
8,7,3,2024-01-17,Back pain physiotherapy,400
10,2,2,2024-01-19,Epilepsy management,850
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

5. Add the prefix "APPT-" to the start

of each data line (not header).

sed '2,\$s/^/APPT-/' appointments.csv

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sed '2, $s/^/APPT-/' appointments.csv
appt_id,patient_id,doctor_id,appt_date,diagnosis,fee
APPT-1,1,1,2024-01-10,Hypertension checkup,500
APPT-2,2,5,2024-01-11,Fever and cold,200
APPT-3,3,3,2024-01-12,Knee pain evaluation,750
APPT-4,4,2,2024-01-13,Migraine treatment,600
APPT-5,5,4,2024-01-14,Acne treatment,300
APPT-6,1,5,2024-01-15,Follow-up general checkup,200
APPT-7,6,1,2024-01-16,Cardiac stress test,900
APPT-8,7,3,2024-01-17,Back pain physiotherapy,400
APPT-9,8,5,2024-01-18,Routine vaccination,150
APPT-10,2,2,2024-01-19,Epilepsy management,850
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

Linux Task 3 - Pipeline challenge

Using single pipeline commands:

1. Find the top 3 highest fee appointments,

show appt_id and fee only.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ sort -t',' -k6 -nr appointments.csv | cut -d',' -f1,6 | head -3
7,900
10,850
3,750
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

2. List unique doctor_ids with

count of their appointments,

sorted highest to lowest.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ tail -n +2 appointments.csv | cut -d',' -f3 | sort | uniq -c | sort -nr
  4
  3 5
  2 3
  2 2
  2 1
  1 4
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

3. Find all appointments where diagnosis

contains the word "pain".

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ grep -i "pain" appointments.csv
3,3,3,2024-01-12,Knee pain evaluation,750
8,7,3,2024-01-17,Back pain physiotherapy,400
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

4. Calculate total fee collected by

extracting the fee column and

summing using awk in a pipeline

SQL PRACTICE

SQL Exercise 1 - CASE statement introduction

1. From employees:

Add a salary_band column using CASE:

salary >= 85000 → "Senior"

salary >= 65000 → "Mid"

salary >= 0 → "Junior"

NULL salary → "Unknown"

Show: name, department, salary, salary_band.

```
7  -- Show: name, department, salary, salary_band.
8  SELECT NAME,
9         DEPARTMENT,
10        SALARY,
11        CASE
12            WHEN SALARY IS NULL THEN 'UNKNOWN'
13            WHEN SALARY >= 85000 THEN 'Senior'
14            WHEN SALARY >= 65000 THEN 'Mid'
15            WHEN SALARY >= 0 THEN 'Junior'
16            ELSE 'UNKNOWN'
17        END AS SALARY_BAND
```

Query result

Script output

DBMS output

Explain Plan

SQL history

  Download ▾ Execution time: 0.007 seconds

	NAME	DEPARTMENT	SALARY	SALARY_BAND
1	Alice Johnson	Engineering	85000	Senior
2	Bob Smith	Marketing	62000	Junior
3	Carol Williams	Engineering	92000	Senior
4	David Brown	HR	55000	Junior

2. From orders:

Add a status_flag column using CASE:

"Delivered" → "Completed"

"Shipped" → "In Transit"

"Pending" → "Waiting"

"Cancelled" → "Closed"

Show: order_id, status, amount, status_flag.

[SQL Worksheet]*

```

1  SELECT * FROM ORDERS;
2  SELECT ORDER_ID,
3         AMOUNT,
4         STATUS,
5         CASE
6             WHEN STATUS = 'Delivered' THEN 'Completed'
7             WHEN STATUS = 'Delivered' THEN 'Completed'
8             WHEN STATUS = 'Shipped' THEN 'In Transit'
9             WHEN STATUS = 'Pending' THEN 'Waiting'
10            WHEN STATUS = 'Cancelled' THEN 'Closed'
11            ELSE 'Unknown'
12        END AS STATUS_FLAG
13 FROM ORDERS;

```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download
Execution time: 0.003 seconds

	ORDER_ID	AMOUNT	STATUS	STATUS_FLAG
1	1001	1299.99	Delivered	Completed
2	1002	59.98	Delivered	Completed
3	1003	349.99	Pending	Waiting

SQL Exercise 2 - String Functions

1. From employees:

Display name in UPPER case.

Display department in LOWER case.

Show the length of each employee's name.

Show first 5 characters of name using SUBSTR.

[SQL Worksheet]*      Aa  

```
1  SELECT
2      UPPER(NAME) AS CAPITAL_LATTER,
3      LOWER(DEPARTMENT) AS SMALL_LATTER,
4      LENGTH(NAME) AS NAME_LENGTH,
5      SUBSTR(NAME, 1, 5) AS NAME_FIRST5
6  FROM EMPLOYEES;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download 

Execution time: 0.008 seconds

	CAPITAL_LATTER	SMALL_LATTER	NAME_LENGTH	NAME_FIRST5
1	ALICE JOHNSON	engineering	13	Alice
2	BOB SMITH	marketing	9	Bob S
3	CAROL WILLIAMS	engineering	14	Carol
4	DAVID BROWN	hr	11	David

2. From customers:

Concatenate first_name and last_name

into a single column called full_name

with a space between them.

Use CONCAT or || operator.

```
11  --      with a space between them.
12  --      Use CONCAT or || operator.
13  SELECT * FROM CUSTOMERS;
14  SELECT
15  |      CONCAT(CONCAT(FIRST_NAME, ' '), LAST_NAME) AS FULL_NAME
16  FROM CUSTOMERS;
17
18  SELECT
19  |      FIRST_NAME || ' ' || LAST_NAME AS FULL_NAME
20  FROM CUSTOMERS;
21
22
23
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.002 seconds

	FULL_NAME
1	James Anderson
2	Priya Sharma
3	Chen Wei
4	Maria Garcia

3. From patients:

Show names where city contains more
than 5 characters.

Use LENGTH().

```
--
22
23  --3. From patients:
24  --  Show names where city contains more
25  --  than 5 characters.
26  --  Use LENGTH().
27  SELECT * FROM PATIENTS;
28  SELECT NAME, CITY FROM PATIENTS WHERE LENGTH(CITY) > 5;
29
30
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.003 seconds

	NAME	CITY
1	Sneha Patel	Mumbai
2	Tom Williams	Chicago
3	Liu Yang	Beijing
4	Ana Gonzalez	Madrid

4. From doctors:

Remove the "Dr. " prefix from name column

in the output. Do not update the table.

Use REPLACE or SUBSTR.

```
29
30 -- 4. From doctors:
31 --     Remove the "Dr. " prefix from name column
32 --     in the output. Do not update the table.
33 --     Use REPLACE or SUBSTR.
34 SELECT * FROM DOCTORS;
35 SELECT
36     REPLACE(NAME, 'Dr. ', '') AS NAME_REPLACE
37 FROM DOCTORS;
38
39
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.006 seconds

	NAME_REPLACE
1	Anil Mehta
2	Sarah Johnson
3	Raj Singh
4	Lisa Park

2. From appointments:

Show doctor_id, count of appointments,

total fee collected, average fee,

and a workload_level column:

count >= 3 → "High"

count = 2 → "Medium"

count = 1 → "Low"

Use HAVING to exclude doctors with

zero appointments.

Order by total fee descending.

[SQL Worksheet]*      Aa 

```
1 SELECT DOCTOR_ID,
2     COUNT(*) AS APPT_COUNT,
3     SUM(FEE) AS TOTAL_FEE,
4     AVG(FEE) AS AVG_FEE,
5     CASE
6         WHEN COUNT(*) >= 3 THEN 'High'
7         WHEN COUNT(*) = 2 THEN 'Medium'
8         WHEN COUNT(*) = 1 THEN 'Low'
9         ELSE 'Unknow'
10    END AS WORKLOAD_LEVEL
11 From APPOINTMENTS
12 GROUP BY DOCTOR_ID
13 HAVING COUNT(*) > 0
14 ORDER BY TOTAL_FEE DESC;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.004 seconds

	DOCTOR_ID	APPT_COUNT	TOTAL_FEE	AVG_FEE	WORKLOAD_LEVEL
1	2	2	1450	725	Medium
2	1	2	1400	700	Medium

PYTHON PRACTICE

Exercise 1 - Pandas filtering and transformation

Read the patients_processed.csv created on Day 8.

If not available, recreate patients.csv and load it.

1. Filter patients where fee is above the

average fee. Do this in one line.

Hint: `df[df['fee'] > df['fee'].mean()]`

```
# 1. Filter patients where fee is above the
#     average fee. Do this in one line.
#     Hint: df[df['fee'] > df['fee'].mean()]

def code_divider():
    print("=====")

import pandas as pd
df = pd.read_csv("patients_processed.csv")
print(df)

code_divider()
average_fee = df[df['fee'] > df['fee'].mean()]
print(average_fee)
```

✓ 0.0s

Output:-

```
=====
   patient_id  name  age blood_type  city  fee fee_category \
0           1  Raj Kumar   34        O+  Delhi   500      Medium
2           3  Tom Williams   46         B-  Chicago   750       High
3           4  Fatima Al-Sayed   29        AB+  Dubai   600      Medium
5           6  Ana Gonzalez   56         A-  Madrid   900       High
...
0      Adult
2      Adult
3      Young
5      Senior
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

2. Filter patients who are either from Delhi

or Mumbai AND have fee above 300.

```
# 2. Filter patients who are either from Delhi
#     or Mumbai AND have fee above 300.
filter_patients_data = df[((df['city'] == 'Delhi') | (df['city'] == 'Mumbai')) & (df['fee'] > 300)]
print(filter_patients_data)
```

✓ 0.0s

Python

```
patient_id  name  age blood_type  city  fee fee_category age_group
0           1  Raj Kumar   34        O+  Delhi   500      Medium      Adult
```

3. Create a new column called `fee_after_discount`:

Senior patients (age ≥ 50) get 20% discount.

All others pay full fee.

Use `apply()` with a lambda or a named function.

```
# using named function
import pandas as pd
df = pd.read_csv("patients_processed.csv")

def senior_patients(row):
    if row['age'] >= 50:
        return row['fee'] * 0.2
    else:
        return row['fee']

# using Lambda
df['fee_after_discount'] = df.apply(senior_patients, axis=1)
print(df)
df['fee_after_discount'] = df.apply(lambda x: x['fee'] * 0.2 if x['age'] >= 50 else x['fee'], axis=1)
print(df)
```

Output:-

	patient_id	name	age	blood_type	city	fee	fee_category	\
0	1	Raj Kumar	34	O+	Delhi	500	Medium	
1	2	Sneha Patel	39	A+	Mumbai	200	Low	
2	3	Tom Williams	46	B-	Chicago	750	High	
3	4	Fatima Al-Sayed	29	AB+	Dubai	600	Medium	
4	5	Liu Yang	24	O-	Beijing	300	Low	
5	6	Ana Gonzalez	56	A-	Madrid	900	High	
6	7	James Okafor	32	B+	Lagos	400	Medium	
7	8	Emily Carter	19	O+	Toronto	150	Low	

	age_group	fee_after_discount
0	Adult	500.0
1	Adult	200.0
2	Adult	750.0
3	Young	600.0
4	Young	300.0
5	Senior	180.0
6	Adult	400.0
7	Young	150.0

4. Find the age group (Young/Adult/Senior from Day 8)

that has the highest average fee.

```
# 4. Find the age group (Young/Adult/Senior from Day 8)
# that has the highest average fee.
df = pd.read_csv("patients_processed.csv")

avg_fee = df.groupby('age_group')['fee'].mean()
result = avg_fee.idxmax()
print(f'\{result}\ Age group is highest average fee.')
```

✓ 0.0s

'Senior' Age group is highest average fee.

5. Create a pivot-style summary using `groupby()`:

For each `age_group` show:

- count of patients
- mean fee
- max fee
- min fee

Use `groupby().agg()` with a dict.

```
# 5. Create a pivot-style summary using groupby():
# For each age_group show:
# - count of patients
# - mean fee
# - max fee
# - min fee
# Use groupby().agg() with a dict.
summary = df.groupby('age_group').agg({
    "patient_id": "count",
    "fee": ["mean", "max", "min"]
})
summary_column = ["count_patients", "mean-fee", "max-fee", "min_fee"]
print(summary)
```

✓ 0.0s

	patient_id	fee		
	count	mean	max	min
age_group				
Adult	4	462.5	750	200
Senior	1	900.0	900	900
Young	3	350.0	600	150

Exercise 2 - Pandas: Reading and handling JSON

Create a file called hospital_config.json:

Write code to:

1. Load the JSON using the json module.

```
# 1. Load the JSON using the json module.
with open("hospital_config.json", "r") as file:
    data = json.load(file)
    print(data)
```

✓ 0.0s

Python

```
[{'hospital_name': 'City General Hospital', 'departments': ['Cardiology', 'Neurology', 'Orthopedics', 'Dermatolo
```

2. Print the hospital name.

```
# 2. Print the hospital name.
print(f"Hospital name is: {hospital_data[0]['hospital_name']}")
```

✓ 0.0s

Hospital name is: City General Hospital

3. Convert the staff list into a pandas DataFrame.

```
# 3. Convert the staff list into a pandas DataFrame
import pandas as pd
df = pd.DataFrame(hospital_data[0]["staff"])
print(df)
```

✓ 0.0s

	id	name	specialty	years
0	1	Dr. Anil Mehta	Cardiology	15
1	2	Dr. Sarah Johnson	Neurology	12
2	3	Dr. Raj Singh	Orthopedics	8
3	4	Dr. Lisa Park	Dermatology	6
4	5	Dr. Carlos Torres	General Medicine	20

4. Add a new column called standard_fee by mapping

each doctor's specialty to fee_structure.

Hint: df['specialty'].map(fee_structure_dict)

```
# 4. Add a new column called standard_fee by mapping
# each doctor's specialty to fee_structure.
# Hint: df['specialty'].map(fee_structure_dict)
fee_structure = hospital_data[0]["fee_structure"]

df["standard_fee"] = df["specialty"].map(fee_structure)
print(df)
```

✓ 0.0s

	id	name	specialty	years	standard_fee
0	1	Dr. Anil Mehta	Cardiology	15	800
1	2	Dr. Sarah Johnson	Neurology	12	700
2	3	Dr. Raj Singh	Orthopedics	8	650
3	4	Dr. Lisa Park	Dermatology	6	400
4	5	Dr. Carlos Torres	General Medicine	20	250

5. Find the doctor with the most experience.

```
# 5. Find the doctor with the most experience.

most_expe_doctor = df.loc[df["years"].idxmax()]

print("most_expe_doctor:")
print(most_expe_doctor)
```

✓ 0.0s

```
most_expe_doctor:
id                5
name             Dr. Carlos Torres
specialty        General Medicine
years            20
standard_fee     250
Name: 4, dtype: object
```

6. Find the average standard_fee across all doctors.

```
# 6. Find the average standard_fee across all doctors.
average = df["standard_fee"].mean()
print(f"Average Fee: {average}")
```

✓ 0.0s

Average Fee: 560.0

7. Add a column called seniority:

years >= 15 → "Principal"

years >= 10 → "Senior"

below 10 → "Associate"

```
# 7. Add a column called seniority:
#   years >= 15 → "Principal"
#   years >= 10 → "Senior"
#   below 10   → "Associate"
def year_band(x):
    if x >= 15:
        return "Principal"
    elif x >= 10:
        return "Senior"
    else:
        return "Associate"

df["seniority"] = df["years"].apply(year_band)
print(df)
# 8. Export the final DataFrame to doctors_info.csv.
```

✓ 0.0s

	id	name	specialty	years	standard_fee	seniority
0	1	Dr. Anil Mehta	Cardiology	15	800	Principal
1	2	Dr. Sarah Johnson	Neurology	12	700	Senior
2	3	Dr. Raj Singh	Orthopedics	8	650	Associate
3	4	Dr. Lisa Park	Dermatology	6	400	Associate
4	5	Dr. Carlos Torres	General Medicine	20	250	Principal

8. Export the final DataFrame to doctors_info.csv.

```
# 8. Export the final DataFrame to doctors_info.csv.  
df.to_csv("doctors_info.csv", index=False)  
  
df = pd.read_csv("doctors_info.csv")  
print(df)
```

✓ 0.0s

	id	name	specialty	years	standard_fee	seniority
0	1	Dr. Anil Mehta	Cardiology	15	800	Principal
1	2	Dr. Sarah Johnson	Neurology	12	700	Senior
2	3	Dr. Raj Singh	Orthopedics	8	650	Associate
3	4	Dr. Lisa Park	Dermatology	6	400	Associate
4	5	Dr. Carlos Torres	General Medicine	20	250	Principal

Exercise 3 - Pandas merge (introduction)

You have two DataFrames.

Create them from the CSV files or hardcode:

df_appointments (from appointments.csv)

df_doctors (from doctors.csv)

Task:

1. Merge appointments with doctors on doctor_id.

Use pd.merge() with how='left'.

2. After merging, the DataFrame should have:

appt_id, patient_id, doctor_id, appt_date,

diagnosis, fee, doctor name, specialty,

experience_years.

```
import pandas as pd

df_appointments = pd.read_csv("appointments.csv")
df_doctors = pd.read_csv("doctors.csv")
df_patients = pd.read_csv("patients.csv")
```

✓ 0.0s

Generate

+ Code

+ Markdown

```
# 1. Merge appointments with doctors on doctor_id.
# Use pd.merge() with how='left'.
merged_df = pd.merge(
    df_appointments,
    df_doctors,
    on="doctor_id",
    how="left"
)
print(merged_df.columns)
```

✓ 0.0s

```
Index(['appt_id', 'patient_id', 'doctor_id', 'appt_date', 'diagnosis', 'fee',
      'name', 'specialty', 'experience_years'],
      dtype='object')
```

3. From the merged DataFrame:

- Find which specialty generated the most total fee revenue.
- Find the average fee per specialty.
- Find which doctor had the most appointments.

```

# 3. From the merged DataFrame:
# - Find which specialty generated
#   the most total fee revenue.
# - Find the average fee per specialty.
# - Find which doctor had the most appointments.

total_fee_revenue = merged_df.groupby("specialty")["fee"].sum()

top_specialty = total_fee_revenue.idxmax()
print(f"Top Revenue Specialty: {top_specialty}")

ave_fee_per_specialty = merged_df.groupby("specialty")["fee"].mean()
print(ave_fee_per_specialty)

appt_count = merged_df.groupby("name")["appt_id"].count()

top_doctor = appt_count.idxmax()
print(f"Doctor with most appointments: {top_doctor}")

```

✓ 0.0s

```

Top Revenue Specialty: Neurology
specialty
Cardiology      700.000000
Dermatology     300.000000
General Medicine 183.333333
Neurology       725.000000
Orthopedics     575.000000
Name: fee, dtype: float64
Doctor with most appointments: Dr. Carlos Torres

```

4. Now merge appointments with patients

on patient_id.

Find the average fee paid per city.

```

# 4. Now merge appointments with patients
#   on patient_id.
#   Find the average fee paid per city.
merged_appt_and_pati = pd.merge(
    df_appointments,
    df_patients,
    on="patient_id",
    how="left"
)
print(merged_appt_and_pati.columns)
average_fee_per_city = merged_appt_and_pati.groupby("city")["fee_y"].mean()

print(average_fee_per_city)

```

✓ 0.0s

```

Index(['appt_id', 'patient_id', 'doctor_id', 'appt_date', 'diagnosis', 'fee_x',
      'name', 'age', 'blood_type', 'city', 'fee_y'],
      dtype='object')
city
Beijing      300.0
Chicago      750.0
Delhi        500.0
Dubai        600.0
Lagos        400.0
Madrid       900.0
Mumbai       200.0
Toronto      150.0
Name: fee_y, dtype: float64

```

#Exercise 4 - *args and **kwargs revision

with data engineering context

Write a function called generate_report that:

- Takes a title as a required argument
- Takes any number of data lines via *args

(each line is a string to print)

- Takes optional formatting via **kwargs:

separator (default "=")

width (default 40)

show_count (default True)

The function should:

- Print the title centered within width chars
- Print separator line of width chars
- Print each data line
- If show_count is True, print total line count

```
def generate_report(title, *args, **kwargs):
    separator = kwargs.get("separator", "=")
    width = kwargs.get("width", 40)
    show_count = kwargs.get("show_count", True)

    print(title.center(width))
    print(separator * width)

    for line in args:
        print(line)

    if show_count:
        print(separator * width)
        print(f"Total lines: {len(args)}")

    # Test with:
    generate_report(
        "Patient Summary",
        "Total Patients : 8",
        "Avg Fee      : 485",
        "Top City      : Madrid",
        separator="-",
        width=45,
        show_count=True
    )
```

Exercise 5 - Lambda and apply() revision

1. Use apply() with lambda to create a column

fee_in_inr converting USD fee to INR

(multiply by 83).

```
# 1. Use apply() with lambda to create a columnS
#   fee_in_inr converting USD fee to INR
#   (multiply by 83).

df["fee_in_inr"] = df["fee"].apply(lambda x: x * 83)
print(df)
```

✓ 0.0s

	name	age	fee	city	fee_in_inr
0	Raj	34	500	Delhi	41500
1	Sneha	39	200	Mumbai	16600
2	Tom	46	750	Chicago	62250
3	Fatima	29	600	Dubai	49800
4	Liu	24	300	Beijing	24900
5	Ana	56	900	Madrid	74700

2. Use apply() with lambda to create a column

name_length with length of each name.

```
# 2. Use apply() with lambda to create a column
#   name_length with length of each name.
|
df["name_length"] = df["name"].apply(lambda x: len(x))
print(df)
```

✓ 0.0s

	name	age	fee	city	fee_in_inr	name_length
0	Raj	34	500	Delhi	41500	3
1	Sneha	39	200	Mumbai	16600	5
2	Tom	46	750	Chicago	62250	3
3	Fatima	29	600	Dubai	49800	6
4	Liu	24	300	Beijing	24900	3
5	Ana	56	900	Madrid	74700	3

3. Use `apply()` with a named function to create

a column called `risk_category`:

age > 50 AND fee > 700 → "High Risk"

age > 40 → "Medium Risk"

otherwise → "Low Risk"

```
# 3. Use apply() with a named function to create
# a column called risk_category:
# age > 50 AND fee > 700 → "High Risk"
# age > 40 → "Medium Risk"
# otherwise → "Low Risk"

def risk_category_band(x):
    if x["age"] > 50 and x["fee"] > 700:
        return "High Risk"
    elif x["age"] > 40:
        return "Medium Risk"
    else:
        return "Low Risk"

df["risk_category"] = df.apply(risk_category_band, axis=1)
print(df)
```

✓ 0.0s

	name	age	fee	city	fee_in_inr	name_length	risk_category
0	Raj	34	500	Delhi	41500	3	Low Risk
1	Sneha	39	200	Mumbai	16600	5	Low Risk
2	Tom	46	750	Chicago	62250	3	Medium Risk
3	Fatima	29	600	Dubai	49800	6	Low Risk
4	Liu	24	300	Beijing	24900	3	Low Risk
5	Ana	56	900	Madrid	74700	3	High Risk

4. Use `apply()` on the entire row (`axis=1`)

to create a column called `profile_tag`

combining name and city:

Example: "Raj | Delhi"

```
# 4. Use apply() on the entire row (axis=1)
# to create a column called profile_tag
# combining name and city:
# Example: "Raj | Delhi"
df["profile_tag"] = df.apply(lambda x: f"{x["name"]} | {x["city"]}", axis=1)
print(df)
```

✓ 0.0s

	name	age	fee	city	fee_in_inr	name_length	risk_category	\
0	Raj	34	500	Delhi	41500	3	Low Risk	
1	Sneha	39	200	Mumbai	16600	5	Low Risk	
2	Tom	46	750	Chicago	62250	3	Medium Risk	
3	Fatima	29	600	Dubai	49800	6	Low Risk	
4	Liu	24	300	Beijing	24900	3	Low Risk	
5	Ana	56	900	Madrid	74700	3	High Risk	

	profile_tag
0	Raj Delhi
1	Sneha Mumbai
2	Tom Chicago
3	Fatima Dubai
4	Liu Beijing
5	Ana Madrid